Beyond the Limits of DPA: Combined Side-Channel Collision Attacks

Andrey Bogdanov and Ilya Kizhvatov

Abstract—The problem of extracting the highest possible amount of key-related information using the lowest possible number of measurements is one of the central questions in side-channel attacks against embedded implementations of cryptographic algorithms. To address it, this work proposes a novel framework enhancing side-channel collision attacks with divide-and-conquer attacks such as differential power analysis (DPA). An information-theoretical metric is introduced for the evaluation of collision detection efficiency. Improved methods of dimension reduction for side-channel traces are developed based on a statistical model of euclidean distance. Experimental results confirm that DPA-combined collision attacks are superior to both DPA-only and collision-only attacks. The new methods of dimension reduction lead to further complexity improvements. All attacks are treated for the case of AES-128 and are practically validated on a widespread 8-bit RISC microcontroller.

Index Terms—Analysis of algorithms and problem complexity, data encryption, cryptographic implementations, physical security, side-channel analysis, collision attacks, AES.

1 Introduction

1.1 Motivation

Keyed cryptographic algorithms employ secret information to protect user data and can provide its confidentiality, integrity, authenticity, and nonrepudiation—services crucial for almost any security-related application. Numerous analysis methods have been proposed for cryptographic algorithms. While the traditional mathematical attacks are solely based on the inputs and outputs of an algorithm to recover its key, side-channel attacks rely upon the fact that any real-world implementation of the algorithm is not ideal and leaks some physically observable parameters that are dependent on the key value processed. Such parameters can include time, power consumption, electromagnetic radiation, or algorithm behavior under actively induced execution faults. Since the attacker often has immediate physical access to embedded systems, they are most vulnerable to side-channel attacks. One of the fundamental problems in side-channel analysis is as follows:

Problem 1 (Fundamental for Side-Channel Analysis). *Extract the highest possible amount of key information given the lowest possible amount of side-channel information for a fixed implementation of a cryptographic algorithm.*

Side-channel collision attacks provide a natural basis for solving this problem, possessing the unique combination of three important properties which are not simultaneously present in any other side-channel analysis technique known today: first, they are essentially based on the *algorithmic properties* of the attacked cryptographic algorithm, which allows the adversary to use more side-channel information from one algorithm execution. Second, they are *not based on any particular leakage model* which opens up the possibility of using relevant side-channel information not limited to a specific model. Third, they do *not require any significant a priori knowledge* of the implementation (a major limitation in many side-channel attacks), however, being able to profit from profiling. Side-channel collision attacks have also further attractive features such as that essential parts of the cryptographic algorithm can remain unknown to the attacker which makes many algorithmic masking techniques transparent to collision attacks.

In this work, we come up with two novel techniques enhancing side-channel collision-based analysis and propose a general framework naturally incorporating them.

1.2 Collision Attacks in the Context

In this section, we aim to draw attention to some of the beneficial features of collision attacks mentioned above that they exhibit in the context of other approaches to side-channel analysis.

Regarding the method of *extracting key-related information*, there are two large classes of side-channel attacks: *leakage-model* oriented and *pattern-matching* oriented. With respect to the *key-recovery procedure*, side-channel attacks fall into two categories: *divide-and-conquer* attacks (which provide distinguishers for small key chunks) and *analytic* attacks (which recover the entire key, e.g., by solving systems of equations). Correspondingly, when classifying according to information extraction method and key-recovery procedure, one can speak about the four types of side-channel attacks represented in Table 1. Note that, optionally, side-channel attacks can use profiling, which we consider in this work, without introducing a separate dimension in the table.

- *A. Bogdanov is with the Katholieke Universiteit Leuven, ESAT/SCD (COSIC), Kasteelpark Arenberg 10 bus 2446, Leuven-Heverlee B-3001, Belgium. E-mail: andrey.bogdanov@esat.kuleuven.be.*
- *I. Kizhvatov is with the Faculty of Science, Technology and Communication, University of Luxembourg, 6, rue Richard Coudenhove-Kalergi L-1359, Luxembourg. E-mail: ilya.kizhvatov@uni.lu.*

Manuscript received 21 Nov. 2010; revised 17 June 2011; accepted 7 July 2011; published online 22 July 2011.
Recommended for acceptance by H. Shen.
For information on obtaining reprints of this article, please send E-mail to: tc@computer.org, and reference IEEECS Log Number TC-2010-11-0642.
Digital Object Identifier no. 10.1109/TC.2011.140.

0018-9340/12/$31.00 © 2012 IEEE Published by the IEEE Computer Society

TABLE 1
Side-Channel Attacks: Methods of Extracting Key-Related Information and Key-Recovery Procedure

	Leakage model	Pattern-matching
Divide-and-conquer	DPA[15] CPA [9] MIA [3]	template [10] stochastic [22]
Analytic	algebraic [21]	collision [6], [23]

Differential power analysis (DPA) [15] and correlation power analysis (CPA) [9], a generalization of DPA, are probably the most widespread practical attacks on numerous embedded systems such as smart-card microcontrollers and dedicated ASICs. They are based on guessing a chunk of the key, classifying traces according to this hypothesis, and performing a statistical test in a leakage model such as Hamming weight or Hamming distance. Statistical tests in DPA attacks are essentially correlation based [18] and thus can capture only linear dependencies between the intermediate variables and the leakage. Similarly to DPA, mutual information analysis (MIA) [3] is based on subkey guessing and classifying traces. However, the test performed for each key hypothesis uses an information-theoretic metric which works with more generic leakage models.

Template attacks [10], [1] belong to another class of powerful side-channel attacks and are optimal in an information-theoretic sense. They do not rely on any particular leakage model but require a profiling stage and, as DPA, are mainly limited to key chunks. Stochastic methods [22], [12] can be seen as a version of template attacks allowing one to simplify template building, further increase the resolution and, thus, decrease the total number of measurements needed.

Algebraic side-channel attacks [21] use Hamming weights of intermediate variables detected by observing side-channel traces to simplify the systems of nonlinear equations on the full key. Thus, algebraic side-channel attacks imply that the implementation leaks Hamming weight related side-channel information.

Side-channel collision attacks [6], [7], [8], [24], [23] use pattern-matching techniques (like template attacks) being however essentially based on the cryptanalytic properties of attacked cryptographic algorithms (by attacking key as a whole as in algebraic side-channel attacks) and not relying on any complex profiling stages (similarly to DPA).

Side-channel attacks with analytic key recovery tend to be more efficient in terms of measurement complexity. Side-channel attacks using pattern-matching information extraction are independent of a concrete leakage model (such as Hamming weight or Hamming distance), thus, being a way more universal. Collision attacks share both these benefits.

Recently, some side-channel techniques using more than one method of extracting key-related information have been proposed. For instance, differential cluster analysis [2] is a divide-and-conquer attack that generally uses pattern matching but can benefit from the knowledge of leakage model. However, these attacks do not use the advantages of analytic key recovery.

At the same time, collision attacks have important limitations that in some cases might make them inapplicable in practice. Namely, they require knowledge of time moments when the leakage of a certain event occurs (e.g., S-box executions), they are more efficient on an 8-bit architecture than on architectures with wider buses and, finally, they are highly sensitive to the false positives of collision detection.

In this work, we show that the analytic key-recovery procedures of collision attacks allow for extensions and propose a general framework for incorporating key-related information resulting from divide-and-conquer attacks such as DPA and template attacks into collision techniques.

1.3 Our Contributions

In this paper, we introduce the *combined collision attack* which is a novel technique for combining side-channel collision attacks with divide-and-conquer attacks such as DPA and template attacks, thus, using *both* divide-and-conquer and analytic key recovery as well as *both* leakage models and pattern-matching extraction (Section 3). This combination of essentially different side-channel techniques allows to use more key-relevant information contained in the side-channel traces, omitted by each of these techniques when applied separately. We theoretically compute the success probability and expected computational complexity of combined collision attacks (Section 4).

Starting from the basic euclidean distance, we propose new techniques of efficient dimension reduction for collision detection. We study some of their statistical properties in a formal way. We propose the usage of *λ-divergence* as a metric for the comparison of different collision detection methods (Section 5).

We practically demonstrate that DPA-combined collision attacks are more efficient than both conventional collision attacks and DPA (Section 6). On the theoretical side, this fact naturally implies that a DPA-combined collision attack uses the key information contained in the traces more efficiently than both a stand-alone conventional collision attack and a stand-alone DPA attack. On the practical side, the new findings allow us to further reduce the measurement complexity of side-channel collision attacks. We conclude with a discussion and open problems in Section 7.

2 Preliminaries

2.1 Internal Collisions

An *internal collision* in a cryptographic algorithm $\mathcal{A}$ occurs with respect to a *target function* ϕ, if ϕ delivers the same output y given two inputs x_1 and x_2: $y = \phi(x_1) = \phi(x_2)$ that are not necessarily equal. If $\mathcal{A}$ is an iterative cipher and ϕ is applied in its first iterations, it can be difficult for the attacker to say if $\phi(x_1) = \phi(x_2)$ using black-box queries (plaintext-ciphertext pairs) only. However, side-channel leakage can help him to detect internal collisions.

Assume that some function ψ processes the output of ϕ. If ϕ returns an equal value y for two inputs, $\mathcal{A}$ performs two identical calculations $\psi(y)$ in these two cases. If ϕ returns two unequal values y_1 and y_2, the corresponding calculations $\psi(y_1)$ and $\psi(y_2)$ are distinct. The attacker can observe this behavior in the power consumption or electromagnetic radiation of the device implementing $\mathcal{A}$ during the application of ψ: similar side-channel traces for equal outputs of ϕ and diverse side-channel traces for unequal outputs of ϕ. Because of this, ψ is called a *collision detection function*.

Once an internal collision is detected, it can be interpreted as a key-dependent equation $\phi(x_1) = \phi(x_2)$ delivering some information about the key, if ϕ and/or x_1 as well as x_2 are key dependent.

However, if there are several applications of ϕ within the algorithm $\mathcal{A}$ and ϕ is invertible, one does not need a separate collision detection function ψ and can employ ϕ as both a target function and a collision detection function. Moreover, a collision now leads to a much simpler algebraic equation $x_1 = x_2$. The latter observation has yielded the idea behind *generalized collisions* proposed in [6] and provides a major advantage over other collision-based attacks such as [23].

2.2 Previous Work on Collision Attacks

The principle of using internal collisions in cryptanalysis is due to Hans Dobbertin and was also discussed in the early work [26]. In [24], a collision attack on DES was proposed, which was enhanced in [16] by almost collisions. In [23], the separate bytes of each of the four 4-byte linear MixColumn mappings in the first AES round are treated as target functions ϕ, S-boxes of the second round representing the collision detection function ψ. In [4], it is shown that similar side-channel collision attacks can be applied to AES-based MACs. The results in [5] suggest that collision attacks can help overpass the random masking of some AES implementations. Overpassing random masking with collision attacks for the case of DES was done in [13] and improved in [14]. In [19], collision attacks were applied to a masked S-box implementation, exploiting the remaining minor leakage. The work [20] also employed pattern matching against a masked and shuffled S-box implementation.

As mentioned above, side-channel collision attacks on AES were improved in [6] by introducing the notion of generalized collisions that occur if two S-boxes at some arbitrary positions of some arbitrary rounds process an equal byte value within several runs. Here, both the target and collision detection functions ϕ and ψ coincide being the 8-bit AES S-box. The S-box remains the same for all executions, rounds, and byte positions within the round (as opposed to DES). This increases the number of function instances to compare, i.e., the number of potential collisions to be used afterward for key recovery.

While Bogdanov [6] treats the *linear collisions* (resulting in linear equations on the key) of AES which are generalized collisions that occur in the first AES round only, the work [8] also considers *nonlinear collisions* (respectively, resulting in nonlinear equations). A set of such collisions can be considered as a system of equations over a finite field. Ways to deal with unreliable collision detection are discussed in [7], including the techniques of binary and ternary voting.

2.3 AES as Target Cryptographic Algorithm

To simplify representation, we chose to study collision attacks at the notable example of the US encryption standard AES [11]. More precisely, we use the key recovery [6] for AES-128 based on linear collisions for this purpose. Note that all techniques of this work can be successfully applied to other ciphers as well as to other collision-based key-recovery techniques.

We use the following notation to represent the variables of AES. $K = \{k_j\}_{j=1}^{16}$, $k_j \in \mathbb{F}_{2^8}$ is the 16-byte user-supplied key (the initial AES subkey). AES plaintexts are denoted by $P^i = \{p_j^i\}_{j=1}^{16}$, $p_j^i \in \mathbb{F}_{2^8}$, where $i = 1, 2, \ldots$ is the number of an AES execution.

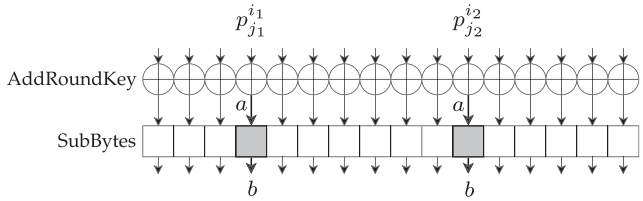

Fig. 1. A linear collision for a pair of AES executions.

2.4 Linear Collision-Based Key Recovery for AES

Given a collision within the first round of AES (linear collision)

$$S\big(p_{j_1}^{i_1} \oplus k_{j_1}\big) = S\big(p_{j_2}^{i_2} \oplus k_{j_2}\big), \tag{1}$$

one obtains a linear equation with respect to the key over $\mathbb{F}_{2^8}$ of the form

$$k_{j_1} \oplus k_{j_2} = p_{j_1}^{i_1} \oplus p_{j_2}^{i_2} = \Delta_{j_1,j_2} \quad \text{for } j_1 \neq j_2. \tag{2}$$

If D collisions have been detected, they can be interpreted as a system of linear equations over $\mathbb{F}_{2^8}$:

$$\begin{cases} k_{j_1} \oplus k_{j_2} & = & \Delta_{j_1,j_2} \\ & \ldots & \\ k_{j_{2D-1}} \oplus k_{j_{2D}} & = & \Delta_{j_{2D-1},j_{2D}}. \end{cases} \tag{3}$$

See Fig. 1 for an illustration.

This system cannot have the full rank due to the binomial form of its equations. Moreover, for small numbers of inputs to AES, the system is not connected and it can be divided into a set of h_0 smaller independent (with disjunct variables) connected subsystems with respect to the parts of the key. Each subsystem has one free variable. Let h_1 be the number of all missing variables, and $h = h_0 + h_1$. We call each of these h subsystems or missing variables a *chain*.

Without loss of generality, a chain ζ of length n can be represented as the following subsystem of the equation system (3):

$$\begin{cases} k_{j_1} \oplus k_{j_2} & = & \Delta_{j_1,j_2} \\ k_{j_2} \oplus k_{j_3} & = & \Delta_{j_2,j_3} \\ & \ldots & \\ k_{j_{n-2}} \oplus k_{j_{n-1}} & = & \Delta_{j_{n-2},j_{n-1}} \\ k_{j_{n-1}} \oplus k_{j_n} & = & \Delta_{j_{n-1},j_n}, \end{cases} \tag{4}$$

or alternatively as an n-tuple of byte indices $\zeta = (j_1, \ldots, j_n)$ in a short form. Each chain (4) has 2^8 possible solutions, since it is sufficient to guess one key byte in the chain to unambiguously determine all other $n-1$ bytes of the chain. If the system (3) has h chains, then it has 2^{8h} solutions.

That is, 2^{8h} guesses have to be performed, which is the offline computational complexity of this basic key-recovery method. Each full key hypothesis is then tested using a known plaintext-ciphertext pair with AES to rule out wrong candidates. Note that 2^{8h} quickly becomes feasible as the number of distinct inputs P^i grows. In a nutshell, it has been shown in [6] that six randomly drawn plaintexts P^i generate enough byte collisions in AES to cover the full 128-bit key by just $h = 5$ or less chains. This drives the offline computational complexity down to at most 2^{40} AES runs.

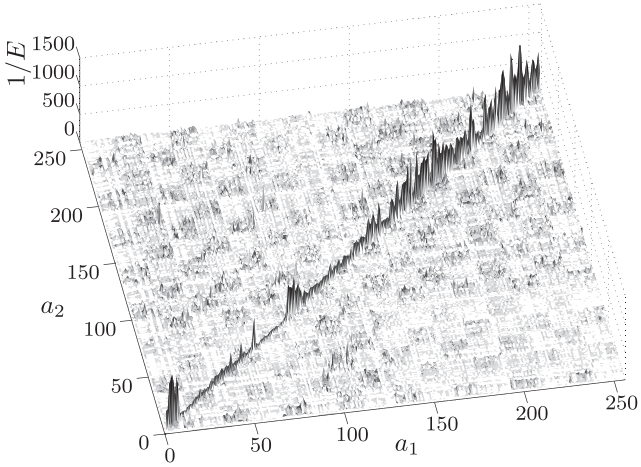

Fig. 2. Inverse euclidean distance $1/E$ between power consumption traces for all input pairs of the AES S-box as implemented on 8-bit RISC μC ATMega16.

2.5 Collision Detection with Euclidean Distance

For a collision attack to be successful, one has to decide if two S-boxes accept equal inputs using side-channel information obtained from the side-channel leakage (of the implementation) of the attacked cryptographic algorithm. Given two side-channel traces

$$\tau_1 = (\tau_{1,1}, \ldots, \tau_{1,l}) \in \mathbb{R}^l \text{ and } \tau_2 = (\tau_{2,1}, \ldots, \tau_{2,l}) \in \mathbb{R}^l,$$

respectively, corresponding to a pair of S-box executions with some unknown inputs a_1 and a_2, it has to be decided whether $a_1 = a_2$ for collision detection.

The two traces can be considered as two vectors in the euclidean space of dimension l. The euclidean distance E between them is defined as

$$E(\tau_1, \tau_2) = \sum_{r=1}^{l} (\tau_{1,r} - \tau_{2,r})^2.$$

One expects that E will be higher for noncollisions and lower for collisions. Our experiments with a popular AVR microcontroller (μC) (see Fig. 2) show that this intuition is indeed justified, at least when noise is somewhat reduced by averaging traces. Note that regular patterns observed in the noncollisions in Fig. 2 are presumably due to a special implementation of the fifth bit of one of the buses in the 8-bit AVR core.

Most papers on collision attacks [23], [6], [7], [8] use the euclidean distance between two noisy traces as the basic metric for collision detection. In [19] and [20], Pearson's correlation was employed. We have found that collision detection can be significantly improved by dimension reduction for side-channel traces based on the properties of the euclidean distance, which is therefore the metric of our choice. We detail this in Section 5.

2.6 Differential Side-Channel Analysis

The technique of differential side-channel analysis was suggested in [15] as DPA and has been extensively studied since then. Now, the name unites several specific attacks that follow the same general scheme. As in the case of collision attacks, here we describe differential side-channel analysis at the example of AES-128.

As already mentioned, differential side-channel analysis is a divide-and-conquer attack. Namely, it recovers each of the AES key bytes $k_1, \ldots, k_{16}$ independently. For each byte k_j, the attack takes a set of known plaintexts P^i and a set of corresponding side-channel traces. The attack algorithms differ between specific attacks, but are all essentially based on comparing the hypothetical leakage of an intermediate key-dependent *target variable* (usually the first round S-box output) predicted for each of the 256 key-byte candidates with the actually observed leakage (represented by the side-channel traces) by means of a specific statistical test, e.g., correlation [9]. Higher values of the statistic suggest that the candidate is more likely to be correct. In the context of this paper, it suffices to represent the output of the differential side-channel attack for a byte k_j of the key as the list $\mathcal{K}_j = (\kappa_j^1, \ldots, \kappa_j^{256})$ of 256 key-byte candidates sorted by the output of the statistical test in the descending order. Then, the correct key-byte candidate is more likely to be closer to the top of the list.

In the simplest case, the attack is limited to considering only the topmost candidate κ_j^1 for each key byte, such that there is a single candidate for the full key. In general, if one considers m topmost candidates for each key byte from the corresponding list, there will be m^{16} full key candidates. An offline exhaustive search among these m^{16} candidates is then necessary to determine the correct one with a plaintext-ciphertext pair at hand. For example, if $m = 5$ candidates for each key byte are considered, the offline computational complexity is about 2^{37} AES runs.

3 Framework for Combined Attacks

In this section, we propose a general framework for the side-channel analysis of cryptographic algorithms based upon internal collisions. This framework allows the adversary to amplify collision attacks by any divide-and-conquer side-channel attack. We refer to such amplified attacks as *combined collision attacks*.

Later, we will study the complexity and success probability of combined collision attack (Section 4) and evaluate them in a practical setting (Section 6).

3.1 The Idea in a Nutshell

In the combined collision attack, we use two building blocks: the linear collision-based key recovery of Section 2.4 and a divide-and-conquer attack, such as DPA of Section 2.6. The major idea is to test chains of key bytes in collision attacks using key-byte ranking from DPA. This reduces the required offline computational complexity of collision-based key recovery.

For instance, if just $h = 5$ chains cover the entire 16-byte key of AES-128, the pure collision attack has to perform $2^{8h} = 2^{40}$ full key tests (see Section 2.4), since all key-byte candidates are equiprobable. However, DPA outputs a ranking of candidates for each key byte, $\mathcal{K}_j = (\kappa_j^1, \ldots, \kappa_j^{256})$ as shown in Section 2.6. So, if this DPA information is available in the collision attack, the adversary can test only the most probable candidates of key-byte chains. If the

number of surviving chain candidates is reduced from 2^8 to 2^4 for each chain, the offline computational complexity will be just $2^{4h} = 2^{20}$ AES runs.

Equivalently, this reduces the online measurement complexity by keeping the budget of offline computational complexity at the level of 2^{40}. We use this fact to address Problem 1.

3.2 Attack Flow

Let the AES-128 implementation have a 16-byte key fixed for the entire attack and leak a key-dependent side-channel parameter (e.g., power consumption or electromagnetic radiation).

A collision attack consists of an online stage, a signal processing stage, and a key-recovery stage. Its procedure is outlined in Algorithm 1 and is explained here:

- In the *online stage* (steps 1-2 of Algorithm 1), N chosen 16-byte plaintexts P^i are sent to the attacked device implementing AES (**ChooseInputs**). The side-channel traces T^i (e.g., power consumption or electromagnetic radiation) are acquired by the measurement equipment (**AcquireTraces**) for these plaintexts. Each trace T^i contains 16 subtraces, one for each S-box:

$$T^i = \{\tau_j^i\}_{j=1}^{16}.$$

  That is, T^i is a set of 16 individual traces τ_j^i for each of the 16 S-box instances in the first AES round. The trace τ_j^i is a real-valued vector of length l, $\tau_j^i \in \mathbb{R}^l$, thus, containing l samples.

  In our attacks, we will send γ randomly drawn 16-byte plaintexts to the AES encryption, each repeated t times, which yields $N = \gamma \cdot t$ traces.
- In the *signal processing stage* (steps 3-6 of Algorithm 1), collisions are detected in the target traces T^i (**DetectCollisions**) and the divide-and-conquer attack is applied to sort the key-byte candidates in each of the 16 byte positions (**SortKeyByte**). Before applying the signal processing, the traces corresponding to each of γ unique plaintexts are averaged t times to decrease noise. The output of the signal processing stage is the set of detected collisions $\mathcal{C}$ containing 4-tuples $(p_{j_1}^{i_1}, p_{j_2}^{i_2}, j_1, j_2)$ and 16 sorted lists $\mathcal{K}$ of 256 key-byte candidates for each of the 16 byte positions. Depending on the measurement setup and implementation, one might choose to perform trace decimation and denoising in this stage.
- In the *key-recovery stage* (step 7 of Algorithm 1), an AES key candidate K' is computed using the list of detected collisions $\mathcal{C}$ and sorted candidates $\mathcal{K}$ (**RecoverKey** detailed in Algorithm 2). Note that **RecoverKey** can return either the right 16-byte key, a wrong 16-byte key, or an empty set of keys $\emptyset$, if no key candidate has passed the final key testing. By π, we denote the success probability of Algorithm 1 which is the probability that **RecoverKey** returns the right key.

Algorithm 1. Collision attack based on linear collisions combined with a divide-and-conquer test for AES-128

1: $\mathcal{P} = (P^1, \ldots, P^N) \leftarrow$ **ChooseInputs**()
2: $\mathcal{T} = (T^1, \ldots, T^N) \leftarrow$ **AcquireTraces**($\mathcal{P}$)
   {*each trace T^i contains 16 subtraces, one for each S-box*}
3: $\mathcal{C} \leftarrow$ **DetectCollisions**($\mathcal{P}, \mathcal{T}$)
   {*each collision in $\mathcal{C}$ is a four-tuple $(p_{j_1}^{i_1}, p_{j_2}^{i_2}, j_1, j_2)$, see (1)*}
4: **for** each k_j of 16 key bytes **do**
5: $\mathcal{K}_j = (\kappa_j^1, \ldots, \kappa_j^{256}) \leftarrow$ **SortKeyByte**($j, \mathcal{P}, \mathcal{T}$)
6: **end for**
   {*now $\mathcal{K} = (\mathcal{K}_1, \ldots, \mathcal{K}_{16})$ contains 16 sorted lists of key byte candidates of length 256 each*}
7: $K' \leftarrow$ **RecoverKey**($\mathcal{C}, \mathcal{K}$)
8: **return** K' as a key candidate

Algorithm 2. Key-recovery **RecoverKey** based on linear collisions and sorted key-byte candidates from a divide-and-conquer test for AES-128

Require: Collisions $\mathcal{C}$ and sorted key-byte candidates $\mathcal{K}$

1: build h chains $\zeta_1, \ldots, \zeta_h$ from collisions $\mathcal{C}$
   {*a chain ζ_i of length n_i is an n-tuple $(j_1, \ldots, j_{n_i})$, see (4)*}
2: **for** each ζ_i of h chains **do**
3: $\mathcal{G}_i \leftarrow \{0, \ldots, 2^8 - 1\}$, i.e., all 2^8 chain guesses
4: **for** each chain guess $g \in \{0, \ldots, 2^8 - 1\}$ **do**
5: **if** not **TestChain**($\zeta_i, g, \mathcal{K}, \mathcal{C}$) **then**
6: remove chain guess g, $\mathcal{G}_i \leftarrow \mathcal{G}_i \backslash \{g\}$
7: **end if**
8: **end for**
   {*now $\mathcal{G}_i$ only contains survived guesses for chain ζ_i*}
9: **end for**
10: unite chain guesses to full key guesses, $\mathcal{G} \leftarrow \cup_{i=1}^{h} \mathcal{G}_i$
   {*$\mathcal{G}$ contains full key guesses that survived chain filtration*}
11: $K' \leftarrow$ **TestKeysWithAES**($\mathcal{G}$)
12: **return** K' as a key candidate

3.3 Combined Key Recovery

The procedure of the combined key recovery is provided in Algorithm 2 and mainly relies on the test of each chain **TestChain** introduced and analyzed in Section 3.4. This is the major advantage of our approach compared to the conventional collision attacks where it is not possible to test the correctness of each chain separately and steps 4-8 of Algorithm 2 are missing. As opposed to that, the availability of divide-and-conquer information in the combined key recovery allows to test for each chain separately which can provide a significant efficiency gain. This can be reflected in the increased success probability of the attack given some measurement complexity or in the reduced measurement complexity given some success probability, thus, delivering a better solution to Problem 1. We will theoretically compute the success probability for a combined attack in Section 4 and practically evaluate it in Section 6.

3.4 Test of Chain

TestChain has as input:

- Chain ζ of length n consisting of key-byte indices $j_1, \ldots, j_n$.

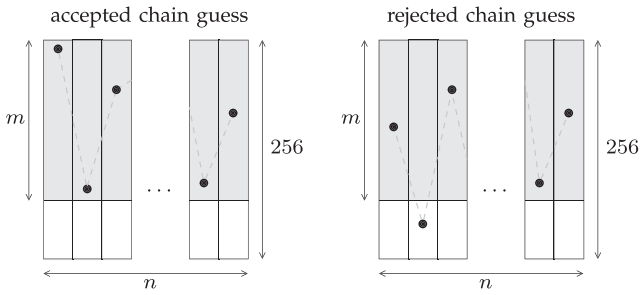

Fig. 3. Test for chain ζ: the guess of chain is rejected if at least one key byte falls outside the most probable m byte values as suggested by the divide-and-conquer test.

- Guess g of the chain. Without loss of generality, we assume that it is the first byte j_1 of the chain and $k_{j_1} = g$.
- The list of (linear) collisions $\mathcal{C}$ to be able to compute all the other $n-1$ key bytes in the chain from k_{j_1}.
- The lists of sorted key-byte candidates $\mathcal{K}$ coming from a divide-and-conquer test (e.g., DPA). Only lists $\mathcal{K}_\ell$ with $\ell \in \{j_1, \ldots, j_n\}$ are needed for the chain to be tested. Each $\mathcal{K}_\ell$ is a sorted list of 256 candidates for the key byte k_ℓ.

The output of **TestChain** is *true*, if the chain ζ_i has passed the test, or *false* otherwise.

The idea of the test of chain is to filter out those guesses of chains that are less probable to be compatible with the key information obtained from a divide-and-conquer test.

In each list $\mathcal{K}_\ell$, we will consider values among the top m positions. These are the most probable candidates for the key byte k_ℓ as suggested by the divide-and-conquer test. We superpose the guess of the chain, computed from the byte guess g for k_{j_1}, and n corresponding sorted lists $\mathcal{K}_\ell$ of length 256 bytes each, see Fig. 3.

Now, the test of chain can be described as follows:

- The guess of chain is accepted if all key bytes of the chain are among the top m candidates, each in the corresponding list $\mathcal{K}_\ell$.
- The guess of chain is rejected if at least one key byte of the chain falls outside the m top candidates in its corresponding list $\mathcal{K}_\ell$.

The threshold m of the test of chain has an impact on the attack complexity and success probability. We detail on this in the next section.

4 Complexity and Success Probability

According to the three stages of a collision attack outlined in Section 3.2, its complexity is defined by three parameters (see Algorithm 1):

- C_{online} is the number of inputs to AES for which measurements have to be performed in the online phase (**AcquireTraces**).
- $C_{\text{processing}}$ is the computational complexity of signal processing on side-channel traces needed to detect collisions (**DetectCollisions**) and sort key-byte candidates within the divide-and-conquer attack (**SortKeyByte**).
- C_{recovery} is the computational complexity of **RecoverKey** (Algorithm 2), that is, the number of operations needed to solve the resulting systems of linear or nonlinear equations and to identify the most probable solution.

For collision attacks in this work, we upper bound C_{recovery} by 2^{40} computations of AES which can be performed within several hours on a PC. Given this restriction on C_{recovery}, the online complexity $C_{\text{online}} = N = t \cdot \gamma$ becomes the major limiting factor of collision attacks, since $C_{\text{processing}}$, mainly determined by γ, will be negligible for our choices of γ.

The success probability π of the attack has to be considered along with C_{online}, since its outcome is often of probabilistic nature. In this work, our main goal is to reduce C_{online} and increase π, given the above admissible upper bound on C_{recovery}.

4.1 Success Probability of Combined Attack

The threshold m of the test of chain described in Section 3.4 has to be chosen in a way that the probability to reject the correct key is small. This probability depends on the distribution of the position of the correct key byte after the divide-and-conquer test used.

Let α be the probability that the correct key byte is among the top m candidates in the sorted divide-and-conquer list. (In terms of the unified framework [25], α is exactly the m-order success rate for a single key-byte recovery.) Under the assumption that all chain tests are independent, the probability for the full correct key to survive after passing the tests with all h chains can be computed as

$$\Pr[\text{correct key survives after } h \text{ chains}] = \prod_{i=1}^{h} \alpha^{n_i} = \alpha^{\sum_{i=1}^{h} n_i} = \alpha^{16},$$

since the sum of all chain lengths is $\sum_{i=1}^{h} n_i = 16$. Moreover, if all collisions have been detected correctly (i.e., collision detection yielded no false positives), this determines the success probability of the full combined collision attack

$$\pi = \alpha^{16}. \tag{5}$$

As a practical example, our experiments with DPA attacks against an AES implementation on the 8-bit ATmega16 μC show that the chances for the correct key-byte guess to be among the top m candidates in the sorted DPA list are quite good already for small values of m (which are preferred to have low C_{recovery}, as we will detail in the next section) and small values of N, i.e., low C_{online}. Fig. 4 depicts the dependency of α upon m for different numbers of inputs N in our DPA attack.

4.2 Complexity of Key Recovery for Combined Attack

Without the test of chain, the complexity of the collision attack is 2^{8h} AES computations, since each chain suggests 2^8 candidates for the respective subset of key bytes and there are h disjunct chains. In the combined approach, we effectively test m candidates for each chain. Moreover, we filter our improbable chain candidates separately for each chain. This results in a lower number of full 16-byte key candidates to be tested with AES at the end which determines C_{recovery}. Since the chain evaluation is much less

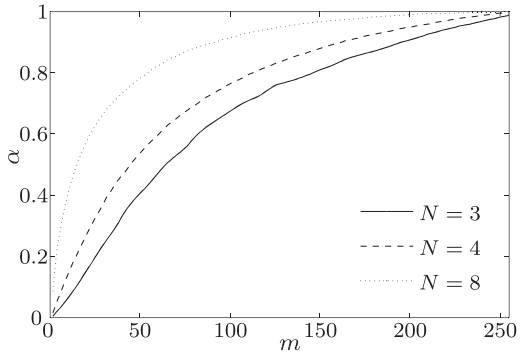

Fig. 4. Empirical dependency of α upon m for different numbers of traces N in a DPA attack.

complex than the testing of a full 16-byte candidate with the full AES, we can win in the total attack complexity significantly. Here, we estimate C_{recovery} given m and α.

The expected number of wrong chain guesses to be tested in the test of chain can be computed as

$$(1-\alpha)m + \alpha(m-1) = m - \alpha.$$

The probability for a wrong chain guess to survive one element of chain can be derived as

$$\alpha\left(\frac{m-1}{255}\right) + (1-\alpha)\left(\frac{m}{255}\right) = \frac{m-\alpha}{255},$$

since a wrong chain guess results in wrong key-byte candidates suggested along the entire chain.

The expected number of correct chain guesses to be tested in the test of chain is α. The probability for a correct chain guess to survive one element of chain is α.

Then, the expected number of chain candidates to survive the test of chain ζ_i can be estimated as

$$\eta_i = (m-\alpha)\left(\frac{m-\alpha}{255}\right)^{n_i-1} + \alpha \cdot \alpha^{n_i-1}.$$

Assuming the independence of all chain tests, we obtain an estimation of the key-recovery complexity for the combined attack:

$$C_{\text{recovery}} \approx \prod_{i=1}^{h} \max(1, \eta_i), \tag{6}$$

where the maximum is taken since one has to test at least one candidate for each chain in practice.

5 Collision Detection

In this section, we propose improved techniques of collision detection by dimension reduction especially tailored for the euclidean metric. The practical evaluation of the dimension reduction techniques in a full (combined) collision attack will be given in Section 6.

5.1 Dimension Reduction in Side-Channel Attacks

Dimension reduction is the selection of samples from side-channel traces, usually with the purpose of improving the efficiency of an attack. In a typical setting, the side-channel trace acquired by the digital oscilloscope at an appropriate sampling rate of at least 10 million samples per second will contain thousands, if not millions, of samples. In the presence of noise, when many traces are required, this makes the trace processing a determining part of the attack complexity and may sometimes even render the attack infeasible. On the other hand, it has long been known that only few samples in a trace would exhibit the leakage, the others being redundant.

Besides the reduced signal processing complexity, another effect of dimension reduction is the potential increase in the attack success rate. The points removed from the side-channel traces would normally carry more noise than the informative signal, while the opposite applies to the selected points. Therefore, dimension reduction would lead to the overall increase in the signal-to-noise ratio (SNR). In most cases, this will lower the number of measurements required to reach a given success rate.

We would like to stress that dimension reduction is not a full-scale profiling; it indeed requires some additional knowledge about the implementation, but this knowledge is less than one would normally impose in traditional profiled attacks to build templates. As opposed to that, in its essence, dimension reduction can be seen as knowledge about the *time moments* when certain computations are carried out, not *values* being processed.

Collision attacks can benefit a lot from the dimension reduction. First, in the following, we show how to select the points from the traces to improve collision detection. We utilize euclidean distance for trace comparison to distinguish between collisions and noncollisions. We also deal with this in the information-theoretic sense by introducing a metric for comparing our dimension reduction techniques. Second, collision detection demands for higher sampling rates (as compared to DPA or other attacks working in the Hamming weight/distance leakage model) [8] to capture subtle differences between the traces, so reduction in the number of samples in a trace is desirable to decrease $C_{\text{processing}}$, which is quadratic in the number of samples and traces.

We start with a statistical model of euclidean distance that we first introduced in [8] and provide here for completeness to support our intuition behind the choice of dimension reduction techniques in the sequel.

5.2 A Statistical Model for Euclidean Distance

Given two traces $\tau_1 = (\tau_{1,1}, \ldots, \tau_{1,l}) \in \mathbb{R}^l$ and $\tau_2 = (\tau_{2,1}, \ldots, \tau_{2,l}) \in \mathbb{R}^l$, we assume that each point $\tau_{i,j}$ can be statistically described as $\tau_{i,j} = s_{i,j} + r_{i,j}$, where $s_{i,j}$ is signal constant (without noise) for the given time point i as well as some fixed input to the S-box, and $r_{i,j}$ is Gaussian noise due to univariate normal distribution[1] with mean 0 and some variance σ^2 remaining the same for all time instances in our rather rough model. Let τ_1 and τ_2 correspond to some S-box inputs a_1 and a_2.

If $a_1 = a_2$, the corresponding deterministic signals are equal (that is, $s_{1,j} = s_{2,j}$ for all js) and one has

$$E(\tau_1, \tau_2)_{a_1=a_2} = \sum_{j=1}^{l} (\tau_{1,j} - \tau_{2,j})^2 = \sum_{j=1}^{l} \xi_j^2 = 2\sigma^2 \sum_{j=1}^{l} \eta_j^2,$$

1. The real measured power consumption is often due to the generic multivariate normal distribution. However, almost all entries of the corresponding covariance matrix are close to zero. Thus, the model with independent multivariate normal distribution seems to be quite realistic.

where $\xi_j = r_{1,j} - r_{2,j}$, $\xi_j \sim \mathcal{N}(0, 2\sigma^2)$, and $\eta_j \sim \mathcal{N}(0,1)$. That is, statistic $E(\tau_1, \tau_2)_{a_1=a_2}$ follows the chi-square distribution with l degrees of freedom up to the coefficient $2\sigma^2$. As the chi-square distribution is approximated by normal distribution for high degrees of freedom, one has the following:

Proposition 1. *Statistic*

$$E(\tau_1, \tau_2)_{a_1=a_2} = \sum_{j=1}^{l} (\tau_{1,j} - \tau_{2,j})^2$$

for $\tau_i = (\tau_{i,1}, \ldots, \tau_{i,l}) \in \mathbb{R}^l$ *with* $\tau_{i,j} \sim \mathcal{N}(s_{i,j}, \sigma^2)$ *can be approximated by normal distribution* $\mathcal{N}(2\sigma^2 l, 8\sigma^4 l)$ *for sufficiently large* l*s.*

Alternatively, if $a_1 \neq a_2$, one has

$$\begin{aligned} E(\tau_1, \tau_2)_{a_1 \neq a_2} &= \sum_{j=1}^{l} (\tau_{1,j} - \tau_{2,j})^2 \\ &= \sum_{j=1}^{l} \left(\delta_j^{(1,2)} + \xi_j\right)^2 \\ &= 2\sigma^2 \sum_{j=1}^{l} \nu_j^2, \end{aligned}$$

where

$$\delta_j^{(1,2)} = s_{1,j} - s_{2,j}, \quad \xi_j = r_{1,j} - r_{2,j},$$
$$\xi_j \sim \mathcal{N}(0, 2\sigma^2) \text{ and } \nu_j \sim \mathcal{N}\left(\delta_j^{(1,2)}/\sqrt{2}\sigma, 1\right).$$

That is, statistic $E(\tau_1, \tau_2)_{a_1 \neq a_2}$ follows the *noncentral* chi-square distribution with l degrees of freedom and $\lambda = \sum_{j=1}^{l} (\delta_j^{(1,2)}/\sqrt{2}\sigma)^2$ up to the coefficient $2\sigma^2$. Again, we have an approximation using

Proposition 2. *Statistic*

$$E(\tau_1, \tau_2)_{a_1 \neq a_2} = \sum_{j=1}^{l} (\tau_{1,j} - \tau_{2,j})^2$$

for $\tau_i = (\tau_{i,1}, \ldots, \tau_{i,l}) \in \mathbb{R}^l$ *with* $\tau_{i,j} \sim \mathcal{N}(s_{i,j}, \sigma^2)$ *can be approximated by normal distribution* $\mathcal{N}(2\sigma^2(l+\lambda), 8\sigma^4(l+2\lambda))$ *with* $\lambda = \sum_{j=1}^{l} (\delta_j^{(1,2)}/\sqrt{2}\sigma)^2$ *for sufficiently large* l*s.*

5.3 Dimension Reduction with Signal Variance (SV)

Signal variance is known [17] to be an adequate indicator of the points leaking information in DPA. We will refer to signal variance as *SV*. More formally, given a fixed level of noise, the points of traces with higher values of $\mathrm{var}(s_{i,j})$ correspond to higher values of SNR [17]. An estimation of variance $\mathrm{var}(s_{i,j})$ is computed for each point j of the trace over input values $a_i \in \mathbb{F}_{2^8}$.

In the sequel, we use SV as a reference dimension reduction technique and show that for collision detection, more efficient criteria exist.

5.4 Dimension Reduction with Signal Difference (SD)

In the comparison of the two traces with the euclidean distance, we try to distinguish between collisions and noncollisions, i.e., between the distributions $E(\tau_1, \tau_2)_{a_1 \neq a_2}$ and $E(\tau_1, \tau_2)_{a_1=a_2}$. As described above, these statistics approximately follow normal distribution for large numbers of trace points. To efficiently distinguish between these two statistics, it is crucial to decrease their variances while keeping the difference of their means high. For this purpose, to better distinguish between collisions and noncollisions, we proposed to discard [8] points of traces with small minimal contribution to the difference of means.

To illustrate this method of dimension reduction, we assume for the moment that $\delta_j^{(1,2)} = 0$ for $j > l/2$ and $\delta_j^{(1,2)} \neq 0$ for $j \leq l/2$ with l even, that is, the second half of the trace does not contain any data dependent information. Then, we can discard the second halves of the both traces τ_1 and τ_2 in the comparison with the euclidean distance and compute two related statistics on the rest of the points:

$$E'(\tau_1, \tau_2)_{a_1=a_2} = \sum_{j=1}^{l/2} (\tau_{1,j} - \tau_{2,j})^2,$$
$$E'(\tau_1, \tau_2)_{a_1 \neq a_2} = \sum_{j=1}^{l/2} (\tau_{1,j} - \tau_{2,j})^2.$$

This will adjust the means and variances of the approximating normal distributions: $\mathcal{N}(\sigma^2 l, 4\sigma^4 l)$ and $\mathcal{N}(2\sigma^2(l/2 + \lambda), 8\sigma^4(l/2 + 2\lambda))$, respectively. Note that the difference of means remains unaffected and equal to $2\sigma^2\lambda$. At the same time, both variances are reduced, one of them by factor 2, which allows one to distinguish between these two distributions more efficiently and, thus, to detect collisions more reliably.

5.4.1 Definition of SD

More generally speaking, for AES, we have to reliably distinguish between inputs in each (a_{i_1}, a_{i_2}) of the $\binom{256}{2}$ pairs of byte values, a_{i_1}, $a_{i_2} \in \mathbb{F}_{2^8}$. Thus, the most informative points j of the traces are those with maximal minimums of $|\delta_j^{(i_1,i_2)}|$ over all pairs of different byte inputs. That is, most informative are points j with maximal values of

$$\min_{a_{i_1} \neq a_{i_2}} |\delta_j^{(i_1,i_2)}| \tag{7}$$

with $\delta_j^{(i_1,i_2)} = s_{i_1,j} - s_{i_2,j}$, where the value of signal $s_{i,j}$ is estimated by trace averaging in practice. In other words, we first fix a trace sample j and then go over all pairs of nonequal inputs. For each pair, we compute the difference between signals. Among those differences, we take the minimum. Trace samples with maximal minimums are expected to contribute more to collision detection. We will denote this point selection criterion as *SD* for brevity.

5.4.2 Comparison to SV

We estimated the values of SD for all time instances j of the S-box lookup in our reference AES implementation and compared this to the signal variance in the same time points, $\mathrm{var}(s_{i,j})$, see Fig. 5a. The figure shows the normalized values of the two criteria (weights) for each point of a side-channel trace of the S-box lookup. One can clearly see that values of each criterion for different points of a trace differ significantly. So, one can select points with the largest weights of a selected criterion, that is, perform dimension reduction of side-channel traces. Fig. 5a reveals that the point selection based on SD is more fine grained than that based on SV. See Section 5.7 for a formal comparison.

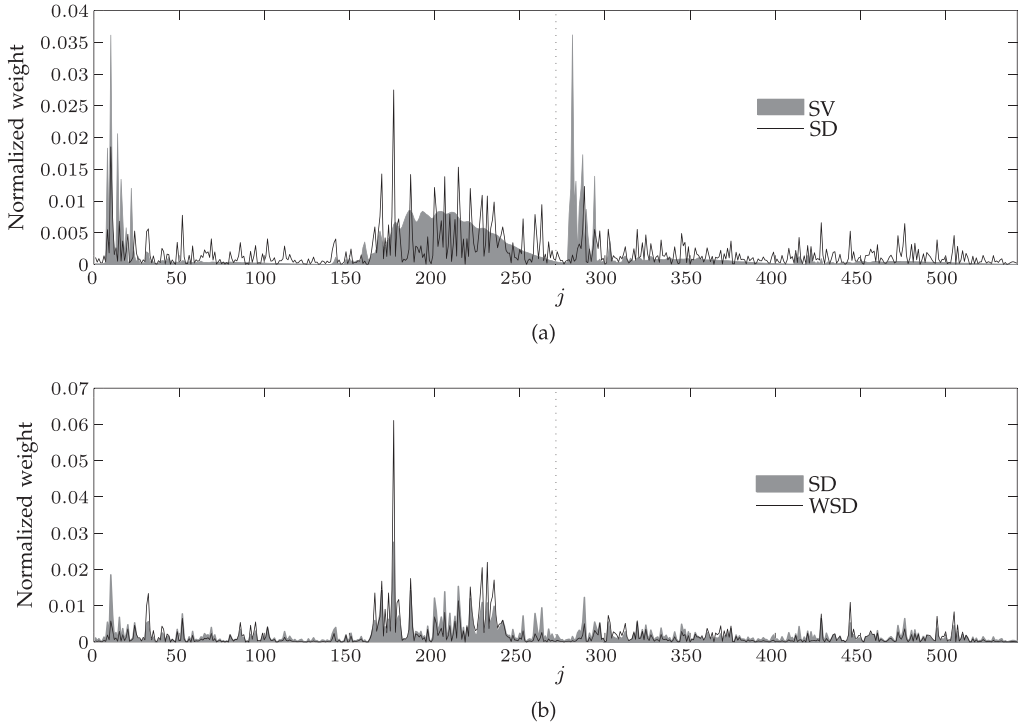

Fig. 5. Different dimension reduction criteria for each point j of a side-channel trace for an AES S-box lookup: (a) Signal difference versus signal variance. (b) Signal difference versus signal difference weighted with noise variance.

5.5 Dimension Reduction with Weighted Signal Difference (WSD)

5.5.1 Definition of WSD

To further amplify the selection of points for collision detection, when calculating the contribution of the point j to the euclidean distance, one can consider not only (7) but also noise variances in this point. Thus, a more efficient criterion, that we call *WSD*, can be defined choosing points that maximize

$$\min_{a_{i_1} \neq a_{i_2}} \frac{|\delta_j^{(i_1,i_2)}|}{\mathrm{var}(r_{i_1,j}) + \mathrm{var}(r_{i_2,j})}$$

with $\mathrm{var}(r_{i,j})$ being the variance of the noise for input i in trace sample j. In practice, the variance is estimated using several traces for a single input. The intuition behind this additional weighting of points by the inverse of the noise variance value is to exclude points that contribute more noise to euclidean distance.

5.5.2 Comparison to SD

In Fig. 5b, we compare WSD to SD for the same two clock cycles of our reference implementation. One can see the differences: weighting by the noise variance increases the contribution of some points while decreasing the contribution of the others (WSD) compared to the pure signal difference. We refer to Section 5.7 for a formal comparison.

5.6 λ-Divergence as Information-Theoretic Metric

As mentioned above, the goal of collision detection is to efficiently distinguish between collisions and noncollisions, that is, between the distribution of the euclidean distance for a pair of equal and nonequal inputs. Here, we propose an information-theoretic measure of difference between these distributions.

Let $\Pr[X]$ be the probability distribution over all possible secret values to be recovered using side-channel information. In the case of collision attacks, X is a set of two elements: $X = \{\text{collision, non-collision}\}$. Further, let $\mathcal{L} = \Pr[L]$ be the probability distribution of side-channel leakage as measured by a collision detection method. For instance, L can be the set of all possible values of the euclidean distance between side-channel traces for pairs of inputs to the S-box. Correspondingly, let $\Pr[L \mid X = x]$ be the probability distribution of leakage, taken separately for collisions and noncollisions, depending on X. We will denote the probability distributions $\mathcal{L}_{\mathrm{C}} = \Pr[L \mid X = \text{collision}]$ and $\mathcal{L}_{\mathrm{N}} = \Pr[L \mid X = \text{non-collision}]$. (Note that in Section 5.2, we have described these distributions in the independent multivariate Gaussian model.)

Our metric uses the notion of *Kullback-Leibler divergence* between two distributions $\mathcal{A}$ and $\mathcal{B}$, $D_{\mathrm{KL}}(\mathcal{A} \parallel \mathcal{B})$. For discrete distributions, it is defined as

$$D_{\mathrm{KL}}(\mathcal{A} \parallel \mathcal{B}) = \sum_i \mathcal{A}(i) \log_2 \frac{\mathcal{A}(i)}{\mathcal{B}(i)}. \tag{8}$$

Note that D_{KL} is not commutative and $D_{\mathrm{KL}}(\mathcal{A} \parallel \mathcal{B}) \neq D_{\mathrm{KL}}(\mathcal{B} \parallel \mathcal{A})$ in general.

To compare collision detection methods, we use the λ-divergence between leakage distributions for collisions and noncollisions which is defined as

$$\begin{aligned} D_\lambda(\mathcal{L}_{\mathrm{C}} \parallel \mathcal{L}_{\mathrm{N}}) = {} & \lambda D_{\mathrm{KL}}(\mathcal{L}_{\mathrm{C}} \parallel \lambda\mathcal{L}_{\mathrm{C}} + (1-\lambda)\mathcal{L}_{\mathrm{N}}) \\ & + (1-\lambda) D_{\mathrm{KL}}(\mathcal{L}_{\mathrm{N}} \parallel \lambda\mathcal{L}_{\mathrm{C}} + (1-\lambda)\mathcal{L}_{\mathrm{N}}), \end{aligned} \tag{9}$$

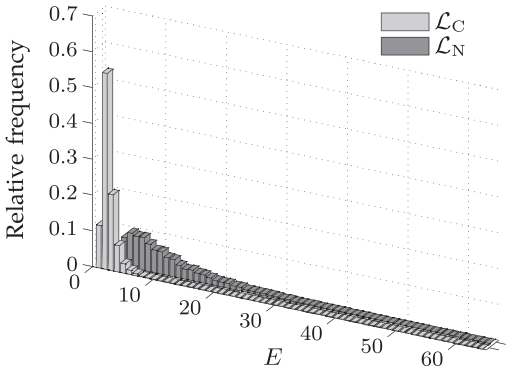

Fig. 6. Empirical distributions for collisions $\mathcal{L}_C$ and noncollisions $\mathcal{L}_N$.

where λ is the a priori probability of a collision, in other words, $\lambda = \Pr[X = \text{collision}]$. Note that we have then

$$\lambda \mathcal{L}_C + (1-\lambda)\mathcal{L}_N = \mathcal{L}.$$

For the 8-bit S-box of the AES, $\lambda = 1/256$.

It is well known that mutual information can be expressed in terms of the Kullback-Leibler divergence. Similarly, it can be shown that the λ-divergence metric as introduced above and the mutual information metric [25] when applied to collision detection instead of template attacks are equivalent.

Proposition 3. $I(X, L) = D_\lambda(\mathcal{L}_C \parallel \mathcal{L}_N)$.

Proof. The left-hand side of the above equality is $I(X, L) = H(L) - H(L \mid X)$ by definition. One can transform the right-hand side using definitions (8) and (9), and knowing that $\lambda = \Pr[X = \text{collision}]$ obtain the left-hand side:

$$\begin{aligned}
&D_\lambda(\mathcal{L}_C \parallel \mathcal{L}_N) \\
&\quad = \lambda D_{KL}(\mathcal{L}_C \parallel \mathcal{L}) + (1-\lambda) D_{KL}(\mathcal{L}_N \parallel \mathcal{L}) \\
&\quad = \lambda \sum_i \mathcal{L}_C(i) \log_2 \frac{\mathcal{L}_C(i)}{\mathcal{L}(i)} + (1-\lambda) \sum_i \mathcal{L}_N(i) \log_2 \frac{\mathcal{L}_N(i)}{\mathcal{L}(i)} \\
&\quad = \left[-\lambda \sum_i \mathcal{L}_C(i) \log_2 \mathcal{L}(i) - (1-\lambda) \sum_i \mathcal{L}_N(i) \log_2 \mathcal{L}(i) \right] \\
&\qquad - \left[-\lambda \sum_i \mathcal{L}_C(i) \log_2 \mathcal{L}_C(i) - (1-\lambda) \sum_i \mathcal{L}_N(i) \log_2 \mathcal{L}_N(i) \right] \\
&\quad = H(L) - H(L \mid X).
\end{aligned}$$

□

So, the information-theoretic metric of the unified framework [25] applies well to the collision detection procedure.

The metric is interpreted in the sense that its higher values (i.e., higher mutual information $I(X, L)$) mean better distinguishing between the distributions and therefore better collision detection. In the following, we use the λ-divergence to compare our collision detection techniques to each other and to the existing techniques.

5.7 Comparison of Collision Detection Techniques with λ-Divergence

For the comparison of collision detection techniques using the λ-divergence, one has to know the distributions $\mathcal{L}_C$ and $\mathcal{L}_N$. In practice, these distributions are estimated empirically.

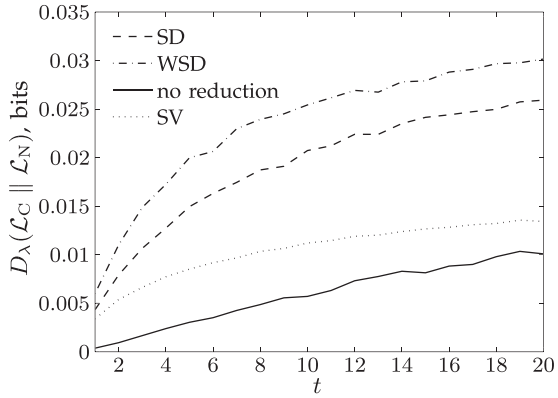

Fig. 7. Effect of dimension reduction on collision detection measured with information-theoretic metric D_λ.

The same problem of estimating the leakage distribution arises in MIA attacks and several distribution estimation methods have been reported to be used [3]. We have opted for the histogram method, that is, we obtain the histograms for $\mathcal{L}_C$ and $\mathcal{L}_N$ from the samplings of euclidean distance using the side-channel traces from our reference implementation for equal and nonequal inputs, respectively. Then, we compute $D_\lambda(\mathcal{L}_C \parallel \mathcal{L}_N)$ following (9), by setting $\lambda = 1/256$.

Fig. 6 shows an example of distributions $\mathcal{L}_C$ and $\mathcal{L}_N$ from our experiments. More precisely, the histogram for $\mathcal{L}_C$ describes the distribution of the euclidean distance between traces corresponding to pairs of colliding inputs, whereas the histogram for $\mathcal{L}_N$ presents the distribution of the euclidean distance between the traces corresponding to pairs of noncolliding inputs. The distributions in this example are visually different.

We have evaluated our two new dimension reduction techniques SD and WSD, the technique SV commonly used in DPA, and detection without dimension reduction as a reference point. We tried averaging t traces to capture the effect of noise reduction.

Fig. 7 presents the experimental results for the considered techniques. The figure demonstrates the gain of collision-related key information brought by one comparison. The horizontal axis shows the data complexity, namely, the number t of trace averagings before comparison. The vertical axis shows information D_λ, measured in bits. Different curves correspond to different dimension reduction methods. Dimension reduction by variance, SV, which works well in DPA attacks, only moderately improves collision detection. Whereas both new methods, SD and WSD, lead to clearly more efficient collision detection, WSD being superior. In other words, they provide lower data complexity for the same amount of information delivered by one comparison. As expected, the information gain grows with the averaging, since the latter increases the SNR.

6 Practical Evaluation

Here, we show that the techniques we have introduced in this work perform well in practice. We implement (linear) collision attacks (Section 2) following our combination framework with the DPA-driven test of chain (Section 3) and employing the new collision detection methods

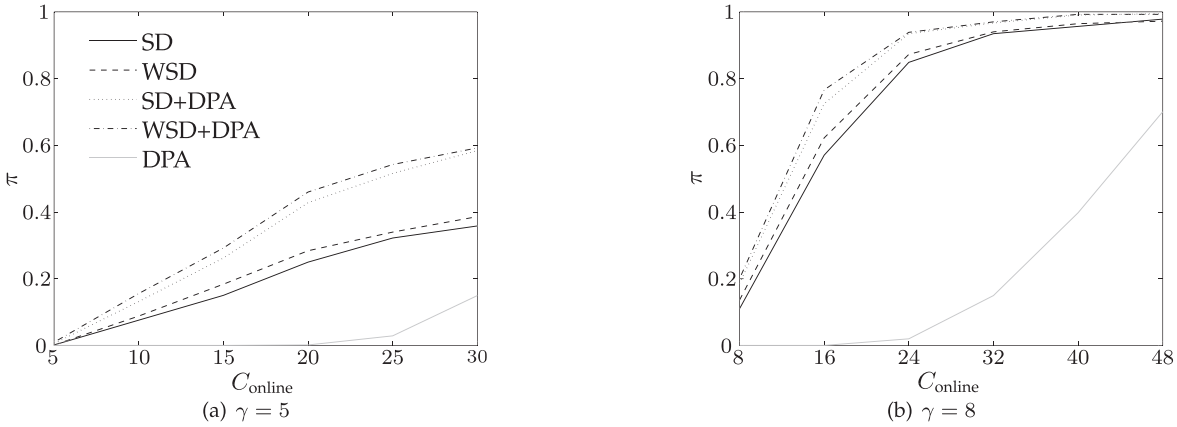

Fig. 8. Attack success rate π in practice against the measurement complexity C_{online} for different number of inputs γ.

(Section 5) against the target implementation. We experimentally estimate the efficiency of these DPA-combined collision attacks.

We performed our attacks for a typical AES-128 implementation on the Atmel ATmega16, an RISC μC from the 8-bit AVR family. We used the standard 256-byte S-box lookup table common for an 8-bit implementation of AES. To run a collision attack, the attacker has to know the time moments when the AES S-boxes are executed. These include instances in SubBytes as well as ShiftRows and MixColumns operations of the AES (the latter involve processing of S-box outputs). We measured the power consumption of the μC while it was encrypting the given plaintexts. The power consumption trace for an S-box lookup execution comprised 4,600 samples. From these, we reduced the dimension to 900 samples using our techniques SD or WSD. This number was chosen empirically by observing when the attack efficiency reached saturation. The component of DPA was implementing CPA in the Hamming weight model [9].

Launching a series of 500 attacks for a given number of inputs γ and averagings t, we experimentally estimated the efficiency parameters defined in Section 4 of all attacks in question. Namely, besides the online complexity $C_{\text{online}} = t \cdot \gamma$, we obtained the success probability π by counting the number of successful attacks: we considered an attack successful if it was possible to recover the correct full AES key with $C_{\text{recovery}} \leq 2^{40}$.

We experimentally characterized four specific attacks employing our techniques: collisions using SD, collisions using WSD, collisions using SD combined with DPA, and collisions using WSD combined with DPA. Comparison of these four attacks in terms of the success rate π of the full key recovery for different number of distinct inputs γ against the total measurement complexity $C_{\text{online}} = t \cdot \gamma$ is presented in Fig. 8. As a reference, we also plot the success rate of the DPA-only attack with $N = C_{\text{online}}$ inputs (note that in a DPA attack, $t = 1$) with C_{recovery} also bounded by about 2^{40}, i.e., when about 2^{40} most probable AES key candidates as suggested by DPA are tested.

One can see that the combination of collision attacks with DPA clearly outperforms both collision-only and DPA-only attacks. The dimension reduction technique WSD outperforms SD, thus, conforming to the information-theoretic comparison in Section 5.7.

7 Conclusions and Open Problems

In this paper, we presented combined divide-and-conquer and collision attacks using side-channel leakage against implementations of cryptographic algorithms. Our experimental results suggest that combined attacks exploit the key information in the side-channel scenario more efficiently than their stand-alone components. Below, we present a relation of our attacks to the existing ones and outline some open problems.

Unlike collision attacks, template attacks require, in addition to a proper dimension reduction, detailed knowledge of the implementation for profiling. This appears to be a much weaker attack model than the one we use. However, the evaluation of template-combined collision attacks using our framework is still an open problem. This combination can possibly reduce the cost of the profiling stage. Another line of future research, initiated in [7], is using profiling to improve collision detection. It is another open problem to evaluate MIA-combined collision attacks using our framework. As in the case of template attacks, we expect this combination to result in a reduced complexity.

While DPA, MIA, and template techniques have been naturally incorporated by the unified framework for comparing side-channel attacks [25], such analytical techniques as collision attacks, considered here, and algebraic attacks [21], cannot be reasonably captured by the unified framework directly. We consider it an important open problem to come up with a further development of the unified framework both practically and generically applicable to analytic attacks. However, in this paper, we successfully applied metrics similar to those in [25] to study some local properties of collision attacks.

From an information-theoretic perspective, each comparison of two traces with the purpose of collision detection should yield in our attacks up to 0.03 bit key-related information (see Fig. 7). However, not all of it is used for key recovery afterward, where only collisions result in equations. At the same time, detected noncollisions also carry useful information ignored by the current techniques. Their usage seems to be technically problematic, since each noncollision would add an equation of a high degree to the system of equations to be solved. We leave this as another open problem.

To prevent attacks we described in this paper, the countermeasures commonly used against side-channel attacks, namely, masking and hiding [17], can be applied. We note that, unlike stand-alone collision attacks,

DPA-combined collision attacks cannot naturally overpass certain masking schemes since masking will disable the DPA component. Overpassing masking might be possible in case higher order DPA [17] or template attacks are used as a divide-and-conquer component of a combined attack. Practicability of such combinations against a masked implementation is an open question.

Acknowledgments

Andrey Bogdanov is a postdoctoral fellow of the Fund for Scientific Research—Flanders (FWO). He was supported in part by a visiting postdoctoral fellow grant within the FWO research project "Linear codes and cryptography" G.0317.06 and by the Research Fund K.U.Leuven grant (OT/08/027) "A mathematical theory for the design of symmetric primitives". The work has been partly sponsored by the IAP Program P6/26 BCRYPT of the Belgian State (Belgian Science Policy) and by the European Commission through the ICT program under contract ICT-2007-216676 ECRYPT II.

References

[1] C. Archambeau, E. Peeters, F.-X. Standaert, and J.-J. Quisquater, "Template Attacks in Principal Subspaces," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '06)*, pp. 1-14, 2006.

[2] L. Batina, B. Gierlichs, and K. Lemke-Rust, "Differential Cluster Analysis," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '09)*, pp. 112-127, 2009.

[3] L. Batina, B. Gierlichs, E. Prouff, M. Rivain, F.-X. Standaert, and N. Veyrat-Charvillon, "Mutual Information Analysis: A Comprehensive Study," *J. Cryptology*, vol. 24, no. 2, pp. 269-291, 2011.

[4] A. Biryukov, A. Bogdanov, D. Khovratovich, and T. Kasper, "Collision Attacks on Alpha-MAC and Other AES-Based MACs," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '07)*, pp. 166-180, 2007.

[5] A. Biryukov and D. Khovratovich, "Two New Techniques of Side-Channel Cryptanalysis," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '07)*, pp. 195-208, 2007.

[6] A. Bogdanov, "Improved Side-Channel Collision Attacks on AES," *Proc. Int'l Conf. Selected Areas in Cryptography (SAC '07)*, pp. 84-95, 2007.

[7] A. Bogdanov, "Multiple-Differential Side-Channel Collision Attacks on AES," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '08)*, pp. 30-44, 2008.

[8] A. Bogdanov, I. Kizhvatov, and A. Pyshkin, "Algebraic Methods in Side-Channel Collision Attacks and Practical Collision Detection," *Proc. Int'l Conf. Cryptology in India: Progress in Cryptology (INDOCRYPT '08)*, pp. 251-265, 2008.

[9] E. Brier, C. Clavier, and F. Olivier, "Correlation Power Analysis with a Leakage Model," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '04)*, pp. 16-29, 2004.

[10] S. Chari, J.R. Rao, and P. Rohatgi, "Template Attacks," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '02)*, pp. 51-62, 2003.

[11] *FIPS, Advanced Encryption Standard. Publication 197*, Nat'l Bureau of Standards, US Dept. of Commerce, 2001.

[12] B. Gierlichs, K. Lemke-Rust, and C. Paar, "Templates vs. Stochastic Methods," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '06)*, pp. 15-29, 2006.

[13] H. Handschuh and B. Preneel, "Blind Differential Cryptanalysis for Enhanced Power Attacks," *Proc. Int'l Conf. Selected Areas in Cryptography (SAC '06)*, pp. 163-173, 2006.

[14] J. Kim, Y. Lee, and S. Lee, "DES with Any Reduced Masked Rounds Is Not Secure against Side-Channel Attacks," *Computers and Math. with Applications*, vol. 60, no. 2, pp. 347-354, 2010.

[15] P.C. Kocher, J. Jaffe, and B. Jun, "Differential Power Analysis," *Proc. Ann. Int'l Cryptology Conf. Advances in Cryptology (CRYPTO '99)*, pp. 388-397, 1999.

[16] H. Ledig, F. Muller, and F. Valette, "Enhancing Collision Attacks," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '04)*, pp. 176-190, 2004.

[17] S. Mangard, E. Oswald, and T. Popp, *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer-Verlag, 2007.

[18] S. Mangard, E. Oswald, and F.-X. Standaert, "One for All - All for One: Unifying Standard DPA Attacks," *IET Information Security*, vol. 5, no. 2, pp. 100-110, June 2011.

[19] A. Moradi, O. Mischke, and T. Eisenbarth, "Correlation-Enhanced Power Analysis Collision Attack," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '10)*, pp. 125-139, 2010.

[20] J. Pan, J.I. den Hartog, and J. Lu, "You Cannot Hide behind the Mask: Power Analysis on a Provably Secure S-Box Implementation," *Information Security Applications*, pp. 178-192, Springer-Verlag, 2009.

[21] M. Renauld, F.-X. Standaert, and N. Veyrat-Charvillon, "Algebraic Side-Channel Attacks on the AES: Why Time Also Matters in DPA," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '09)*, pp. 97-111, 2009.

[22] W. Schindler, K. Lemke, and C. Paar, "A Stochastic Model for Differential Side Channel Cryptanalysis," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '05)*, pp. 30-46, 2005.

[23] K. Schramm, G. Leander, P. Felke, and C. Paar, "A Collision-Attack on AES: Combining Side Channel- and Differential-Attack," *Proc. Int'l Workshop Cryptographic Hardware and Embedded Systems (CHES '04)*, pp. 163-175, 2004.

[24] K. Schramm, T.J. Wollinger, and C. Paar, "A New Class of Collision Attacks and Its Application to DES," *Proc. Int'l Workshop Fast Software Encryption (FSE '03)*, pp. 206-222, 2003.

[25] F.-X. Standaert, T. Malkin, and M. Yung, "A Unified Framework for the Analysis of Side-Channel Key Recovery Attacks," *Proc. Ann. Int'l Conf. Advances in Cryptology: The Theory and Applications of Cryptographic Techniques (EUROCRYPT '09)*, pp. 443-461, 2009.

[26] A. Wiemers, "Collision Attacks for Comp128 on Smartcards," *Proc. ECC-Brainpool WorkshopSide-Channel Attacks on Cryptographic Algorithms*, Dec. 2001.

Andrey Bogdanov studied information security and theoretical computer science at the Russian State University for the Humanities in Moscow and Humboldt-University of Berlin. He received the PhD degree in electrical engineering and computer science from Ruhr University Bochum, Germany, in 2009. He is currently an FWO-supported postdoctoral researcher at the Department of Electrical Engineering of Katholieke Universiteit Leuven in Belgium. In 2004, he was with Siemens AG, Corporate Technology IC 3, Security, in Munich, Germany. From 2005 to 2009, he was a senior security engineer at escrypt GmbH - Embedded Security in Bochum, Germany. In 2011, he was a visiting researcher with Microsoft Research, Cryptography, in Redmond. He holds an IACR Best Paper Award for his work on efficient implementations of postquantum signature schemes (CHES '08). He proposed the first attack on the KeeLoq automotive authentication system—a major and widely used solution for keyless entry. He has codesigned several cryptographic algorithms including the German IT Security Award winning lightweight block cipher PRESENT which is filed to become an ISO encryption standard. He is leading the Cryptographic Algorithms and Protocols work group within the Belgian Fundamental Research Network on Cryptology and Information Security (BCRYPT). His current research interests include symmetric-key cryptographic algorithms and their cryptanalysis, provable security, physical cryptanalysis, as well as efficient cryptographic arithmetics.

Ilya Kizhvatov at the time of writing this paper had just received the PhD degree in computer science from the University of Luxembourg, where he has been doing research in physical attacks on cryptographic algorithm implementations since 2007. He received the diploma with highest honors in information security in 2004 from the Russian State University for the Humanities, Moscow. From 2003 to 2005, he was an embedded software engineer at Neo-Silicon, Moscow, implementing 802.11 protocol stack for ARM-based wireless SoCs.